Conversational ERP Architecture (English)

> **AI Replaces Training + Full CRUD via Natural Language** erpilot's core differentiation — this document = North Star

Version: v1.0 (2026-05-15) · **Status**: Final pre-Phase-1 launch

---

📑 Contents

1. Vision
2. Current Gap: 1.5 / 8
3. 6 Fatal Disasters of Naive Design
4. 6-Layer Architecture
5. 7 Core Design Principles
6. 4-Phase Roadmap
7. Success KPIs
8. Risks & Mitigation
9. Code-Level Mapping

---

1. Vision

> **Employees don't need to know ERP's menus / buttons / fields. They say what they want; AI translates language into system actions. Training = teaching how to talk to the AI, not how to use the ERP UI.**

This inverts the entire software-design tradition — from "design UI for operators" to "design the system for conversation". SAP / Odoo / Workday cannot do this. This is what makes erpilot truly differentiated.

1.1 Customer-Felt Difference

Traditional ERP	erpilot Conversational
1-3 months employee training	**2 hours to proficiency** — knowing how to talk to AI is enough
Find feature = remember menu N levels deep	"I want to do X" — AI takes you (or just does it)
Look up jargon in manual	"What is 3-way match?" — AI explains in plain language
Wrong operation = stuck	"Cancel last action" — undo within 5 minutes
Reports month-end only	"Today's status?" — ask anytime, get answer anytime

---

2. Current Gap

Capability	UI (12 pages)	Natural Language (26 AI tools)
Read	✅ list pages	✅ all 26 tools are read
Create	✅ forms	❌ **0 tools**
Update	❌ **missing**	❌ **0 tools**
Delete	❌ **missing**	❌ **0 tools**

Completion = 1.5 / 8 = 19%

This document's purpose: **close the remaining 81% in the right direction** — not just add tools, but redesign the conversation flow.

---

3. 6 Fatal Disasters of Naive Design

If we just wrote `delete_customer` / `update_inventory` tools and handed them to the AI, 6 disasters would occur. **These 6 dictate our architecture**:

#	Disaster	Example	Fix (in §5)
1	**Hallucination**	AI invents `part_no = "M6-NEW"` that doesn't exist in DB	Principle #1 Risk Tier + #4 Disambiguation
2	**Ambiguity**	"Modify that order" — which order?	Principle #4 Disambiguation
3	**No Confirm**	"Delete customer A" — AI deletes immediately	Principle #2 Confirmation Card
4	**No Undo**	Delete → can't recover	Principle #5 Undo Token
5	**Missing Slots**	"Send PO to ZhongGang" — qty? unit price? AI guesses	Principle #3 Slot-filling
6	**Privilege Escalation**	Sales rep places NT$1M order via AI (has no such permission normally)	Principle #7 RBAC × AI

---

4. 6-Layer Architecture

Each layer fixes one naive disaster. End-to-end flow:

```
│   Session remembers "last action" = PO-2026-105              │
│   Cross-session: owner's frequent queries / company terms    │
│   Fixes: Conversation actually flows like a conversation     │
└──────────────────────────────────────────────────────────────┘
```

---

5. 7 Core Design Principles

Principle #1: Tool Risk-Tier Three Categories

Every tool registered MUST declare `risk_tier`:

Tier	Examples	Behavior
read	list_parts / query_inventory	Execute directly, no confirm
soft-write	create_draft_po / save_search_filter	Execute directly + offer easy undo
hard-write	approve_po / delete_customer / post_journal	**MUST** return Confirmation Card

Principle #2: Confirmation Card Pattern

Hard-write tool's AI **does not execute**. It returns JSON to frontend:

```
{
  "type": "confirm_required",
  "summary_zh": "建立 PO-105 給中鋼 · M6-BOLT-20 × 1000 · NT$500",
  "summary_en": "Create PO-105 to ZhongGang · M6-BOLT-20 × 1000 · NT$500",
  "action": "create_purchase_order",
  "args": { "supplier_id": "S-001", "items": [...] },
  "undo_eligible": true,
  "buttons": ["confirm", "adjust", "cancel"],
  "expires_at": "2026-05-15T10:30:00Z"
}
```

Frontend renders as inline card; user clicks confirm → backend uses `args` to actually call tool.

Principle #3: Slot-filling

Every hard-write tool has a `required_slots` list. AI asks back when slots are missing:

```
User: "Send PO to ZhongGang"
AI: "M6-BOLT-20? M6-NUT? Which one?"  (disambiguation)
User: "M6-BOLT-20"
AI: "How many?"  (missing slot: qty)
User: "1000"
AI: "OK, creating PO for 1000 M6-BOLT-20 to ZhongGang at $0.5/unit
     (last price), total $500. Confirm?"
```

Implementation: Tool registry declares `slots`; AI uses chain-of-thought in LLM prompt to find missing.

Principle #4: Disambiguation Flow

When entity-resolver finds >1 candidate for a single name:

```python
async def _create_po(db, user, supplier: str, items: list[dict]):
    candidates = await search_supplier(db, supplier)
    if len(candidates) > 1:
        return {
            "type": "disambiguation",
            "candidates": [{"id": s.id, "label": s.name} for s in candidates],
            "prompt": f"'{supplier}' matches {len(candidates)} suppliers, which one?",
        }
    ...
```

Principle #5: Undo Token Mechanism

Every soft/hard-write tool writes an `ActionHistory` row:

```
{
  "id": "ah-uuid",
  "session_id": "...",
  "user_id": "...",
  "tool": "create_purchase_order",
  "args_before": {},           # for undo
  "args_after": {...},
  "undo_recipe": "delete_purchase_order(po_id=...)",
  "executed_at": "...",
  "expires_at": "...",  # 5 min later
  "undone": false
}
```

User: "undo last action" → AI finds last ActionHistory in session, not expired → executes undo_recipe.

Principle #6: Training-Replacement Triad

glossary tool (jargon in plain language):

```
User: "What is 3-way match?"
AI: "3-way match = PO vs supplier's delivery note vs our receiving report;
     all 3 must agree on qty/amount before payment. We auto-match and
     push notifications when anomalous."
```

workflow guide tool (process navigation):

```
User: "How do I close the month?"
AI: "7 steps: ① freeze yesterday's transactions ② run AR aging ③ run AP aging
     ④ run inventory valuation ⑤ compute P&L ⑥ manager review ⑦ close month.
     Start from step 1?"
```

learn-our-term tool (company-specific vocabulary):

```
User: "We call M6 BOLT 20mm 'plum-small' internally"
AI: "Got it, your company term: 'plum-small' = M6-BOLT-20. Next time say
     'plum-small inventory' and I'll understand directly."
Writes to TermAlias table.
```

Principle #7: RBAC × AI Integration

Each tool registered with `required_permission`:

```python
@register_tool(
    domain="purchase",
    risk_tier="hard-write",
    required_permission="purchase.order.create",
    slots=["supplier", "items"],
)
async def create_purchase_order(...): ...
```

AI in Layer 3 checks `user.has(tool.required_permission)`; rejects if no permission: "You don't have permission to create POs, please contact your manager."

---

6. 4-Phase Roadmap

Phase 1 (Week 1) — Foundation

Goal: Conversational architecture skeleton running, first hard-write tool e2e demo-able.

See `CONVERSATIONAL_ERP_PHASE1_SPEC_EN.md`.

Day	Work	Deliverable
1	Tool registry + risk-tier framework	`app/agents/registry.py`
2	`ConfirmCardResponse` schema + frontend ConfirmCard component	schema + React component
3	First hard-write tool: `create_purchase_order_with_confirm`	tool + dialogue e2e
4	Slot-filling reverse-ask mechanism	LLM prompt strategy
5	E2E demo: "order 1000 M6 for me" full workflow + recording	demo video

Phase 2 (Week 2) — 16 Core Write Tools

Domain	Write Tools
Inventory	adjust_stock / create_part / update_safety_stock
Purchase	create_po / approve_po / cancel_po
Sales	create_so / confirm_so / ship_so
Production	create_wo / release_wo / complete_wo
Quality	create_capa / mark_ncr_resolved
Accounting	post_journal / close_month

Half day per tool (framework already built → linear investment).

Phase 3 (Week 3) — Conversational Intelligence

- Disambiguation proactive ask
- Glossary tool (200 entries: company + universal ERP jargon)
- Workflow guide tool (10 typical workflows: month-end / stocktake / RMA / ...)
- Undo / rollback: 5-minute window
- Session memory: "last one", "previously mentioned"
- TermAlias personalized vocabulary learning

Phase 4 (Week 4) — Scale + Launch

- Personalization (owner morning briefing / sales quick-reply)
- Company-term learning (teach once, remember forever)

- Mobile conversational interface integration
- Frontend 12 pages add Edit/Delete buttons (safety net for power users who don't want to chat)
- Phase 1-3 full e2e demo

---

7. Success KPIs

MVP KPIs (4 weeks out)

Metric	Target	Measurement
Natural language success rate	"Intent → AI correct execution" ≥ 85%	100 golden queries regression test
Avg training time per employee	< 2 hours	Demo to 5 non-tech employees, time to proficiency
CRUD coverage	All 12 domains × CRUD = 48 cases runnable	E2E test matrix
Misoperation interception	100% hard-write through confirm card	Full e2e tests
Customer onboarding time	< 1 day	New customer onboarding duration

Long-term KPIs (post v2.0)

Metric	Target
AI operation ratio of total	> 60%
Avg user request duration	< 30s (incl. confirm)
Proactive push vs customer asks	> 3:1 (AI thinks ahead)

---

8. Risks & Mitigation

Risk	Severity	Mitigation
LLM hallucination causes data corruption	🔴 Fatal	Confirmation Card + Risk-tier + Undo three-layer defense
LLM cost explosion	🟠 High	Three-tier routing (rule 40% + ollama 50% + cloud 10%) + rate limit
Users resist "confirm first" (too slow)	🟡 Medium	Tier design: read executes directly, soft-write also direct, only hard-write confirms
Prompt injection escalation	🔴 Fatal	Existing Prompt Safety + RBAC × AI dual-check
Conversational state confusion (multi-turn clash)	🟡 Medium	Session 30 min TTL + explicit "reset" command
Multi-language mix (Chinese + English)	🟢 Low	LLM natively supports

9. Code-Level Mapping

New Files

```
backend/app/agents/
├── registry.py                    NEW · Tool risk-tier registry
├── confirm_card.py                NEW · ConfirmCardResponse schema
├── slot_filling.py                NEW · missing-slot reverse-ask
├── disambiguation.py              NEW · ambiguity resolver
├── action_history.py              NEW · undo / rollback
└── domains/
    ├── purchase_write_tools.py    NEW · Phase 2 write tools
    ├── sales_write_tools.py       NEW
    └── ...

backend/app/models/
├── action_history.py              NEW · ORM model for undo
└── term_alias.py                  NEW · company term learning

frontend-desktop/src/components/
├── ConfirmCard.tsx                NEW · confirm card component
├── DisambiguationCard.tsx         NEW · multi-choice card
└── ChatMessage.tsx                MODIFIED · inline card support

frontend-desktop/src/pages/Chat.tsx   MODIFIED · conversation state machine

docs/
├── CONVERSATIONAL_ERP_DESIGN_ZH.md
├── CONVERSATIONAL_ERP_DESIGN_EN.md   (this file)
├── CONVERSATIONAL_ERP_PHASE1_SPEC_ZH.md
└── CONVERSATIONAL_ERP_PHASE1_SPEC_EN.md
```

Modified Files

- `app/api/chat.py` — add ConfirmCard handling path
- `app/agents/engine.py` — inject risk-tier gating
- `app/core/security.py` — RBAC × AI integration
- `app/models/ai_governance.py` — DecisionLog gets confirm_token / undo_token

📎 Related Documents

- Phase 1 Day-1 to Day-5 Spec
- Agent Catalog (existing 26 tools)
- Architecture Blueprint
- **Chinese version**: `CONVERSATIONAL_ERP_DESIGN_ZH.md`

Version: v1.0 · **Last updated**: 2026-05-15 · **Status**: Final pre-Phase-1 launch